

Praxis: Platform-Treiber und Device Tree

Ein vorhandenes Platform Device vom Live Tree über sysfs bis zum gebundenen Treiber verfolgen

4.06

Vier Ergebnisse machen den Treiberpfad nachvollziehbar

01 Kandidat wählen

Ein gebundenes Platform Device mit Device-Tree-Knoten finden.

02 Knoten zuordnen

of_node vom sysfs-Gerät bis in den Live Tree verfolgen.

03 Match belegen

compatible, modalias und driver-Symlink zusammenführen.

04 Ressourcen prüfen

MMIO, IRQ und weitere Angaben mit dem Knoten vergleichen.

Beobachten zuerst · Änderungen nur optional an ausdrücklich unkritischen Geräten

Fünf Werkzeuge verbinden Live Tree, Gerät und Treiber

Werkzeug	Form	Verwendung
Platform sysfs	Geräte, Treiber und Symlinks	Instanz und Bindung finden
Live Device Tree	Knoten und Property-Dateien	Hardwarebeschreibung lesen
dtc	Baum dekompilieren	Zusammenhang durchsuchen
modinfo / Source	Aliase und Match-Tabelle	Treiberunterstützung belegen
dmesg / tracefs	Probe und Bindung beobachten	Zeitlichen Ablauf prüfen

Jede Quelle beantwortet eine andere Frage – zusammen ergeben sie den vollständigen Gerätepfad.

Der Laborablauf folgt einem Gerät durch sechs Stationen



Auswahl

Nur Geräte mit of_node und driver verwenden

Zuordnung

sysfs-Pfad und Device-Tree-Knoten verbinden

Beleg

compatible, modalias und Ressourcen vergleichen

Sicherheit

Der Standardversuch bleibt vollständig read-only.

Fünf Vorbedingungen machen das Labor reproduzierbar

Prüfung	Befehl	Erwartung
Device Tree vorhanden	<code>test -d /sys/firmware/devicetree/base</code>	Verzeichnis existiert
Platform Bus	<code>test -d /sys/bus/platform/devices</code>	Geräte sind sichtbar
dtc verfügbar	<code>command -v dtc</code>	Pfad zum Compiler
Kernel-Log lesbar	<code>dmesg tail</code>	Meldungen oder Hinweis
Build-Tree optional	<code>test -d /lib/modules/\$(uname -r)/build</code>	Quellzugriff möglich

Das Labor funktioniert am besten auf einem DT-basierten ARM- oder RISC-V-System mit installierten dtc-Werkzeugen.

Ein kleines Skript findet geeignete Kandidaten

```
$ PLAT=/sys/bus/platform/devices

$ for d in "$PLAT"/*; do
    test -L "$d/of_node" || continue
    test -L "$d/driver" || continue
    basename "$d"
done | head

$ read -r -p "Device: " NAME
$ DEV="$PLAT/$NAME"
```

- | | | |
|-----------|---------------------|--|
| 01 | Platform Bus | enthält registrierte Platform Devices |
| 02 | of_node | verweist auf den Device-Tree-Knoten |
| 03 | driver | zeigt den aktuell gebundenen Treiber |
| 04 | Auswahl | keine Konsole, Timer oder Boot-Medien wählen |
| 05 | DEV | vollständigen sysfs-Pfad speichern |

Wähle einen Kandidaten, dessen Ausfall das System nicht beeinträchtigen würde – auch wenn der Standardpfad nur liest.

Drei Symlinks verbinden die Perspektiven

sysfs-Gerät

- basename "\$DEV"
- cat "\$DEV/uevent"
- Geräteinstanz
- und modalias



of_node

- readlink -f
- "\$DEV/of_node"
- Knotenpfad
- und Properties



driver

- readlink -f
- "\$DEV/driver"
- Treibername
- und Busbindung

Der Gerätepfad ist der Anker; of_node liefert die Beschreibung und driver die gebundene Implementierung.

Fünf Regeln halten das Live-System sicher

read-only beginnen

Zuerst nur lesen, auflösen und dokumentieren.

kritische Geräte ausschließen

Konsole, Storage, Watchdog und Systemtimer meiden.

kein blindes Unbind

Treiberbindung nur nach Wirkungsanalyse ändern.

Vorher und nachher loggen

Zeitpunkt und Kernelmeldungen festhalten.

Befehle protokollieren

Gerät, Pfade und Ausgaben reproduzierbar sichern.

Ein Diagnose-Labor ist erfolgreich, wenn der Zusammenhang belegt ist – nicht wenn Hardware zwangsläufig umgebunden wurde.

Property-Dateien brauchen typgerechte Darstellung

```
$ NODE=$(readlink -f "$DEV/of_node")
$ tr -d '\0' < "$NODE/compatible"; echo
$ od -An -tx4 "$NODE/reg"
$ od -An -tx4 "$NODE/interrupts" 2>/dev/null
$ tr -d '\0' < "$NODE/status" 2>/dev/null
$ echo
```

- | | | |
|----|-----------------------|--|
| 01 | Strings | compatible ist NUL-separiert |
| 02 | Zellen | reg und interrupts sind Binärwerte |
| 03 | Endianness | od zeigt zunächst rohe 32-Bit-Zellen |
| 04 | Parent-Kontext | #address-cells bestimmt die Interpretation |
| 05 | status | fehlend bedeutet häufig verfügbar |

Property-Dateien nicht pauschal mit cat lesen: Typ und Binding bestimmen die korrekte Interpretation.

dtc macht aus dem Live Tree eine durchsuchbare Ansicht

Live Tree

Root-Pfad

Knotenpfad

compatible

dtc -l fs -O dts

Quelle: Live Tree
Ziel: Textdatei

Auswertung

live-tree.dts

rekonstruierte Baumansicht

Fundstelle

Knoten und Parent-Beziehungen

Der Export zeigt den aufgelösten Laufzeitbaum; Includes, Labels und Kommentare des Original-DTS sind nicht erhalten.

Drei Indizien belegen das Matching

```
$ cat "$DEV/modalias"
$ DRIVER=$(basename "$(readlink -f "$DEV/driver")")
$ echo "$DRIVER"
$ readlink -f "$DEV/of_node"
$ modinfo "$DRIVER" 2>/dev/null | grep '^alias:'
```

01 modalias

kodiert die Geräteidentität für die Modulsuche

02 driver-Symlink

nennt den tatsächlich gebundenen Treiber

03 modinfo / Source

zeigt Aliase; Built-ins im Source Tree prüfen

Ein Beleg ist stark, wenn compatible, modalias, Match-Tabelle und gebundener Treiber zusammenpassen.

Ressourcen aus sysfs und Device Tree gemeinsam prüfen

```
$ cat "$DEV/resource" 2>/dev/null
$ cat "$DEV/uevent"
$ grep -i "$(basename "$DEV")" /proc/iomem
$ grep -i "$DRIVER" /proc/interrupts
$ od -An -tx4 "$NODE/reg"
```

1

MMIO

resource und /proc/iomem vergleichen

2

IRQ

/proc/interrupts mit DT-Spezifikation abgleichen

3

Identität

uevent und modalias zum Gerät sichern

4

Quervergleich

DT-Werte, Kernelresource und Treiber müssen passen

Abweichungen können durch Adressübersetzung, Interrupt-Controller und die Interpretation des jeweiligen Bindings entstehen.

Fünf Nachweise bilden den vollständigen Befund

Nachweis	Quelle	Aussage
Gerätepfad	Platform sysfs	konkrete Kernelinstanz
DT-Knoten	of_node-Symlink	Hardwarebeschreibung
compatible	Property-Datei	unterstützte Gerätefamilie
Treiber	driver-Symlink	tatsächliche Bindung
Ressourcen	resource und /proc	MMIO und IRQ

Der Befund ist vollständig, wenn jede Aussage einen reproduzierbaren Pfad oder Befehl als Nachweis besitzt.

Vier Bedingungen begrenzen optionales Unbind und Bind

01 Gerät ist unkritisch

Kein Boot-Medium, keine Konsole und kein Systemtimer.

02 Nutzung ist ausgeschlossen

Keine offenen Handles, Mounts oder aktiven Verbraucher.

03 Treibername ist bestätigt

bind und unbind gehören zum verifizierten Driver-Pfad.

04 Recovery ist vorbereitet

Lokaler Zugriff und Neustartmöglichkeit sind vorhanden.

Der Kernversuch bleibt read-only; Umbinden ist eine optionale Erweiterung für geeignete Testhardware.

Vier Ergebnisse schließen das Labor ab

```
$ printf 'Device: %s\n' "$DEV"
$ readlink -f "$DEV/of_node"
$ tr -d '\0' < "$NODE/compatible"; echo
$ cat "$DEV/modalias"
$ readlink -f "$DEV/driver"
$ cat "$DEV/resource" 2>/dev/null
```

1

Knoten

sysfs-Gerät und of_node eindeutig dokumentieren

2

Matching

compatible, modalias und Treiber verbinden

3

Ressourcen

MMIO und IRQ nachvollziehbar vergleichen

4

Fazit

Belege, Unsicherheiten und Sicherheitsgrenzen notieren

Nächste Einheit

Auswertung: Gerätepfad, Matching und Ressourcen gemeinsam dokumentieren